INDEX

1. ABSTRACT **1**

2. INTRODUCTION **2**

3. SYSTEM ANALYSIS **3**

3.1 EXISTING SYSTEM 3

3.2 PROPOSED SYSTEM 3

3.3 SYSTEM REQUIREMENTS 4

4. SOFTWARE DEVELOPMENT **6**

4.1 SOFTWARE ENVIRONMENT 6

4.2 DEPLOYMENT 6

5. SYSTEM DESIGN **8**

5.1 SOFTWARE ARCHITECTURE 8

5.2 DATAFLOW DIAGRAM 9

5.3 DATABASE DESIGN 10

5.4 MODULE 12

6. SYSTEM DEVELOPMENT **15**

6.1 INPUT AND OUTPUT DESIGN 15

6.2 SYSTEM STUDY 16

6.3 SYSTEM TESTING 17

7. SAMPLE OUTPUT **19**

8. CONCLUSION **27**

9. BIBLIOGRAPHY **28**

AI-POWERED MULTI-MODAL MEMORY ASSISTANT WITH USER PANEL AND ADMIN CONSOLE

CHAPTER 1
ABSTRACT

Students and professionals today capture knowledge across many scattered tools - voice notes on a phone, lecture slides in cloud storage, screenshots, meeting recordings and handwritten to-do lists - but no single place exists where all of it can be captured, organised, recalled and administered. Conventional note-taking applications store text but cannot connect related ideas, remind the user at the right moment, or give an administrator any visibility into how the platform is being used. The project titled "Pathii: AI Memory Assistant" addresses this gap by building a complete, production-ready web platform that combines a multi-modal user panel with a full administration console.

The system is implemented as a Next.js 16 application with twenty-four pages organised into two cooperating interfaces. The user panel provides a personal knowledge dashboard: voice capture with a live waveform, a searchable and filterable memories library, tasks and schedule management, a monthly calendar, document and people directories, integration shortcuts, an insights dashboard with charts, and personal settings. The admin console provides platform governance: a system overview, user management, memory and document administration, task and event monitoring, AI usage analytics, integration management, platform analytics, report generation, a notification centre, a system health monitor, a roles-and-permissions matrix and global settings. Every page ships with working controls - live search, filter pills, tab bars, checkboxes, toggle switches, dropdowns, add and delete actions, CSV export and toast confirmations - and the interface is responsive from a 607-pixel side pane to a full 1920-pixel desktop.

The frontend was developed through a mockup-driven, review-gated process: each of the twenty-four pages was first transcribed pixel-exactly from an approved design reference, then independently reviewed, visually tested against its reference screenshot, made fully interactive, stripped of demonstration data for a clean production launch, and finally seeded with a small, consistent sample dataset so every screen demonstrates real behaviour. The production build statically pre-renders all routes, contains no external image dependencies, and runs with a single command.

The architecture is designed to connect to a planned FastAPI backend backed by PostgreSQL for relational records, Qdrant for vector similarity search, Neo4j for the knowledge graph and Redis for caching, with Qwen3 as the language model, BGE-M3 for embeddings, Whisper for speech-to-text and Qwen2.5-VL for vision. This report presents the analysis, design, implementation and testing of the platform as built.

Keywords: AI memory assistant, knowledge management, Next.js, React, admin console, retrieval-augmented generation, vector search, responsive web application.

CHAPTER 2
INTRODUCTION

Human memory is a poor storage system for the volume of information a modern student or professional touches every day. A single week can produce lecture recordings, meeting notes, document drafts, contact details, deadlines and half-formed ideas, each trapped in a different application. When the information is needed again, the user must remember which tool held it, what it was called and where in that tool it was filed. Studies of personal knowledge management consistently show that captured information loses most of its value simply because it cannot be found again at the right moment.

An AI memory assistant is a system built around the opposite promise: capture anything, in any format, and let the system organise and resurface it. Speech is transcribed automatically, images are understood by a vision model, text is embedded into a vector space, and retrieval-augmented generation (RAG) answers natural-language questions over the user's own knowledge. Entities such as people, projects and places are linked into a knowledge graph so that related memories surface together. This project, "Pathii: AI Memory Assistant", implements the complete client platform for such a system and prepares the integration points for the AI services.

The platform is organised as two panels with one codebase. The user panel gives each person a private workspace: a dashboard of what happened today, a voice-capture studio, the memories library, tasks and schedule, a calendar, documents, people, integrations, insights and settings. The admin console gives a platform administrator global control: user accounts and roles, memory and document administration, AI usage and platform analytics, notification broadcasting, system health monitoring and configuration. Separating the two audiences lets each interface stay simple while sharing the same design system, navigation architecture and component idioms.

Development followed a strict quality process. Every page began as a fixed design reference; the implementation was transcribed from it and independently reviewed for pixel fidelity. A visual QA pass then screenshot every route at 1920 x 1080 and compared it against its reference image. Interactivity was added in a second phase - search, filters, toggles, checkboxes, CSV export and toast notifications - with hydration safety enforced throughout (no unseeded randomness or live clocks in any render path). A production-readiness pass removed all demonstration records so the application launches empty by default with clean empty states, and a final pass seeded a small, cross-consistent sample dataset so that every screen can be demonstrated meaningfully.

The result is a fully working, responsive, production-built web application: twenty-four routes, statically pre-rendered, with zero external asset dependencies, ready to be connected to the planned FastAPI + PostgreSQL + Qdrant backend described in the system design chapter.

CHAPTER 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing scenario, users rely on a collection of general-purpose tools: the phone's voice recorder for spoken notes, a note-taking app for text, cloud drives for documents, the calendar app for deadlines and a messaging app for everything else. There is no single entry point for capture, no shared organisation and no unified retrieval. Administrators of classroom or team environments have an even weaker position - consumer tools offer no admin console at all, so there is no visibility into usage, storage or content, and no way to manage members, roles or platform policy.

Retrieval is keyword-based at best. A voice note is searchable only by its typed filename, an image only if it was manually tagged, and a half-remembered conversation not at all. Nothing connects the same person, project or topic across different notes. Reminders exist only in the calendar, disconnected from the material they refer to.

Drawbacks of the Existing System

- Capture is fragmented across many tools, so recall requires remembering where something was saved.
- Voice, image and scanned content cannot be searched because they are never transcribed or indexed.
- No semantic connection between related memories; each item is an isolated island.
- No administrative layer: user, storage, usage and content governance is impossible.
- No analytics or insights about how knowledge accumulates over time.
- Data is scattered across personal accounts with no central backup or export.

Problem Statement

Users need a single, multi-modal workspace that captures voice, text, images and documents, organises them automatically, resurfaces them through search and insights, and manages deadlines - and platform owners need an administrative console with full visibility and control over users, content, AI usage and system health.

3.2 PROPOSED SYSTEM

The proposed system, Pathii, is a two-panel web platform built on Next.js. The user panel turns capture into a single action: voice recordings are taken in an in-page studio, notes and files are attached to memories, tasks and events are managed beside them, and an insights dashboard summarises activity. The admin console provides the governance layer over the same platform: user accounts with plans, statuses and teams; memory, document and task administration; AI usage and analytics; a notification broadcaster; a system health monitor; and a roles-and-permissions matrix.

The system is production-ready as a frontend platform: it statically pre-renders all routes, ships with working interactive controls on every page, hides scrollbars for a clean application feel, works from mobile-class widths to full desktop, and contains no external asset dependencies. It is designed to

attach to a FastAPI backend with PostgreSQL, Qdrant, Neo4j and Redis, and to the Qwen3 / BGE-M3 / Whisper / Qwen2.5-VL model stack for transcription, embedding, vision and retrieval.

Advantages

- Single multi-modal capture point for voice, text, images, documents and events.
- Instant client-side retrieval: live search and type filters across every library.
- A complete admin console - users, content, AI usage, notifications, health and roles.
- Responsive from a 607 px pane to a 1920 px desktop with no clipped or overlapping content.
- Production build with static pre-rendering, favicons, a styled 404 page and clean metadata.
- Empty-by-default architecture with consistent empty states and seeded sample data for demos.

Features

- Voice capture studio - waveform, recording state, recent captures with tags.
- Memories library - type filters (voice, note, image, document), live search, detail panel.
- Tasks and schedule - priority tasks, completion checkboxes, category tags, timeline.
- Calendar - month grid, event add, today highlight.
- Documents and people directories - type filters, tags, contact cards.
- Insights - stat cards and charts (area, bar, donut) built with recharts.
- Admin console - 13 pages covering users, memories, documents, tasks, AI usage, integrations, analytics, reports, notifications, system health, roles and settings.
- Working controls everywhere - search, filters, tabs, checkboxes, toggles, dropdowns, add and delete actions, CSV export and toast confirmations.

3.3 SYSTEM REQUIREMENTS

The system was developed and tested on a standard personal computer. Because the delivered application is a statically pre-rendered frontend, it runs on any machine with a JavaScript-capable browser and can be served by any static host or a single Node process.

Hardware Configuration

Component	Requirement
Processor	Any modern dual-core CPU or better (Intel Core i5 or equivalent recommended).
Main Memory (RAM)	4 GB minimum; 8 GB recommended for the development build.
Storage	About 500 MB free space (Node.js, project dependencies and the production build).
System Type	Laptop / desktop computer; the app itself runs in any modern browser.
Input Devices	Keyboard and mouse; microphone for voice capture.
Output Devices	Monitor / display for the dashboards, charts and tables.

Functional Requirements

- Capture voice notes, text notes, images and documents from the user panel.
- Organise memories by type and tag; live search across titles and content.

- Manage tasks with priorities, completion state and categories; manage calendar events.
- Provide document and people directories with type and tag filters.
- Provide an insights dashboard with charts over the user's activity.
- Provide an admin console: manage users, roles, memories, documents, tasks, integrations and notifications; monitor AI usage, analytics, reports and system health.
- Export any table to CSV from the browser.
- Behave correctly on first launch with empty states, and with seeded sample data for demonstrations.

Non-Functional Requirements

- Responsive layout from 607 px to 1920 px with no clipping or overlap; hidden scrollbars.
- Hydration-safe server-side rendering: no unseeded randomness or live clocks in any render path.
- Static pre-rendering of all 24 routes for fast loads and cheap hosting.
- Zero external asset dependencies (avatars are rendered as initials; no third-party image hosts).
- Consistent visual system: Inter typography, one palette, shared card and toast idioms.

CHAPTER 4
SOFTWARE DEVELOPMENT

4.1 SOFTWARE ENVIRONMENT

Category	Details
Operating System	Windows 10 / 11 (development); the production build runs on any OS with Node.js.
Runtime	Node.js 20+ with npm.
Programming Language	TypeScript 5 across all application code.
Framework	Next.js 16.2.12 (App Router) with React 19.2.4 client components.
Styling	Tailwind CSS 4 utility classes with an Inter font loaded through next/font.
Icons	lucide-react icon set (brand icons rendered as inline SVG).
Charts	recharts 3 (area, line, bar and donut charts on the analytics pages).
Development Tools	Visual Studio Code, Git and GitHub for version control.
Build Output	Static pre-rendered pages served by next start or any static host.

Role of the Main Tools

- **Next.js App Router** - file-system routing: every folder under app/ becomes a route; twenty-four page modules map to twenty-four URLs.
- **React client components** - every page is a "use client" component holding its own state: search queries, filter selections, checkboxes, toggles and toast messages.
- **Tailwind CSS 4** - utility-first styling with responsive variants (max-lg:, lg:) layered onto the frozen desktop classes so small screens reflow without changing the desktop render.
- **lucide-react** - a consistent icon vocabulary across both panels.
- **recharts** - declarative charts with gradient fills, tooltips and legends.
- **next/link** - client-side navigation through the sidebar of every page.
- **next/font** - self-hosted Inter typography with no external font requests.

4.2 DEPLOYMENT

The application is started with two commands. Development runs a hot-reloading server; production builds a static bundle and serves it.

```
# development
cd frontend
npm install
npm run dev        # http://localhost:3000

# production
cd frontend
npm run build      # compiles and pre-renders all routes
npm start          # serves the production build on port 3000
```

- **Local** - any laptop or desktop with Node.js 20+; no GPU or database required.

- **Cloud** - the production build can be served by any Node host or static CDN (Vercel, Netlify, a Docker container) because every route is statically pre-rendered.
- **Configuration** - next.config.ts disables the development indicator and pins the Turbopack root; globals.css hides all scrollbars while keeping scroll behaviour.
- **Data** - the delivered build seeds a small sample dataset in browser state; the planned FastAPI backend replaces this with persistent storage.

Steps to Run

- Install Node.js 20 or later.
- Run npm install inside the frontend directory.
- Run npm run dev for development, or npm run build followed by npm start for production.
- Open http://localhost:3000 for the user panel and http://localhost:3000/admin for the console.

Troubleshooting

- If the port is busy, stop the older process or set the PORT environment variable.
- If a page shows stale content after pulling changes, delete the .next folder and rebuild.
- If charts appear mid-animation in automated screenshots, wait for the two-second animation window before capturing.

4.2.1 Project Structure

```
AI-Memory-assistant/
+-- frontend/                   the Next.js application
|   +-- app/
|   |   +-- page.tsx                user dashboard (route /)
|   |   +-- voice-capture/          voice studio
|   |   +-- memories/               memories library (primary design)
|   |   +-- memories-classic/       memories library (alternate design)
|   |   +-- tasks/  calendar/  documents/  people/
|   |   +-- integrations/  insights/  settings/
|   |   +-- admin/                  the admin console
|   |   |   +-- page.tsx            admin overview (/admin)
|   |   |   +-- users/  memories/  documents/  tasks/
|   |   |   +-- ai-usage/  integrations/  analytics/  reports/
|   |   |   +-- notifications/  system-health/  roles/  settings/
|   |   +-- layout.tsx              root layout, Inter font, metadata
|   |   +-- globals.css             theme + hidden scrollbars
|   |   +-- not-found.tsx           styled 404 page
|   |   +-- icon.svg                Pathii brain favicon
|   +-- next.config.ts  package.json
+-- admin/                      13 admin design references (PNG + TSX)
+-- refernce/                   11 user-panel design references (PNG + TSX)
+-- docs/                       architecture, API, database and plan documents
+-- backend/                    planned FastAPI services
```

Each page module is intentionally self-contained: its data, its sidebar markup and its handlers live in one file, which keeps every screen independently testable and reviewable. The two reference folders preserve the approved design sources next to the code that implements them.

CHAPTER 5
SYSTEM DESIGN

5.1 SOFTWARE ARCHITECTURE

Pathii follows a page-modular architecture in which every route is an isolated React client component. There is no shared runtime state between pages; navigation between them is standard client-side linking, and each page mounts with its own seeded state. The architecture has five layers:

- **Routing layer** - the Next.js App Router maps file-system folders to the twenty-four URLs.
- **Page modules** - one self-contained "use client" component per route holding data arrays, state hooks and handlers.
- **Navigation chrome** - every page renders its own sidebar (twelve items in the user panel, twelve plus a roles entry in the console) with the current item highlighted.
- **Interactive layer** - derived visible-row arrays (search + filters), controlled inputs, toast feedback and CSV export helpers.
- **Planned service layer** - REST endpoints on a FastAPI gateway backed by PostgreSQL, Qdrant, Neo4j and Redis for persistence, vector search, graph queries and caching.

5.1.1 User Panel Architecture

The user panel is organised as a dashboard shell: a fixed sidebar, a header with global search and notifications, and a responsive content grid. The root container uses min-h-screen with lg:h-screen lg:overflow-hidden so the desktop keeps its fixed-viewport dashboard behaviour while small screens scroll naturally. Content grids collapse from three or four columns to one below the lg breakpoint, and wide tables and pill rows either wrap or scroll inside their cards. The voice-capture studio is a dedicated page with a recording state machine (idle, recording, paused), a canvas-free CSS waveform and a recent-captures list that grows as the user records.

5.1.2 Admin Console Architecture

The console shares the shell pattern with its own twelve-item navigation plus a thirteenth roles-and-permissions entry reachable from that page. Table pages follow one idiom: a toolbar (search, filter pills, dropdowns, export), a stat-card row derived from the table data, the data table with row checkboxes and actions, and a pagination footer that reports the visible count. Analytics pages compose recharts visualisations over small seeded series; the system-health page renders a service status table, a log feed and a seeded activity heatmap.

5.1.3 State and Data Flow

Each page owns a typed data array (for example five users with plan, status and team fields) and a set of React useState hooks: a search query, an active filter key, selection sets and a toast message. A derived visible-rows expression applies the search and the active filters; the table, the stat cards and the pagination footer all read from that single derived array, so filtering one place updates everything coherently. Add and delete actions mutate the data array immutably, which makes every table feature (search, filter, select, export, pagination) work on user-created rows with no extra

wiring.

5.1.4 Determinism and Hydration Safety

Because every page is statically pre-rendered, the server HTML and the first client render must be byte-identical. The codebase forbids Math.random(), Date.now() and new Date() in any render path. Where the design called for pseudo-random visuals (the analytics and system-health heatmaps), a seeded mulberry32 generator with a constant seed produces identical output on the server and the client. Locale-sensitive number formatting is pinned to en-US. Timestamps in seeded data are static strings such as "Today" and "Yesterday".

5.1.5 Production Chrome

The application ships with a Pathii brain-mark favicon (app/icon.svg), a title template ("%s | Pathii"), a theme colour, a styled 404 page linking back to both panels, and a README with run instructions. The development indicator is disabled and scrollbars are hidden globally while scroll behaviour is preserved.

5.2 DATAFLOW DIAGRAM

The Level-1 data flow diagram below shows how data moves through the platform, including the planned AI processing and storage services.

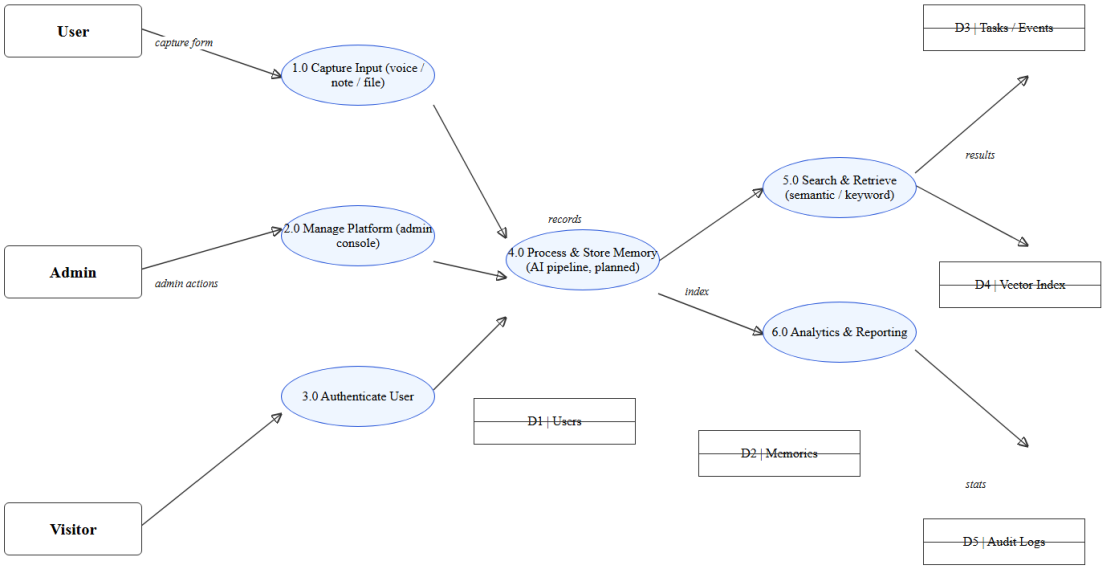


Fig. 5.1 - Level 1 Data Flow Diagram for Pathii

The user submits captures to process 1.0 and personal-management actions (tasks, events, documents) to the same flow; the administrator acts on process 2.0 for platform governance. Process 4.0 represents the planned AI pipeline - transcription, embedding and knowledge-graph construction - writing to the memory store (D2) and the vector index (D4). Process 5.0 answers search queries from the index, and process 6.0 aggregates both stores into the analytics and

reporting dashboards.

Item	Name	Role
1.0	Capture Input	Receives voice notes, text, images and files from the user
2.0	Manage Platform	Admin console actions on users, content and configuration
3.0	Authenticate User	Sign-in and session handling (planned with the backend)
4.0	Process & Store Memory	Transcription, embedding and graph linking (AI pipeline)
5.0	Search & Retrieve	Keyword and semantic search over the vector index
6.0	Analytics & Reporting	Aggregates memories, usage and audit data into dashboards
D1-D5	Data stores	Users, memories, tasks/events, vector index, audit logs

5.3 DATABASE DESIGN

Database table structure for Pathii (PostgreSQL, proposed). The current implementation keeps records in browser state; the schema below is the target design for the FastAPI backend.

#	Field Name	Data Type	Constraint	Key	Description
1	user_id	SERIAL	NOT NULL	PK	Unique identifier for each account
2	name	VARCHAR(100)	NOT NULL	-	Full name
3	email	VARCHAR(150)	NOT NULL	UNIQUE	Sign-in e-mail
4	plan	VARCHAR(20)	NOT NULL	-	free / pro / team
5	status	VARCHAR(20)	NOT NULL	-	active / inactive / suspended
6	team	VARCHAR(60)	NULL	-	Team or department
7	role_id	INT	NOT NULL	FK	References role(role_id)
8	created_at	TIMESTAMP	NOT NULL	-	Account creation time

Table: memory - captured knowledge items (user panel core entity)

#	Field Name	Data Type	Constraint	Key	Description
1	memory_id	SERIAL	NOT NULL	PK	Unique identifier
2	user_id	INT	NOT NULL	FK	Owner of the memory
3	type	VARCHAR(20)	NOT NULL	-	voice / note / image / document / link
4	title	VARCHAR(200)	NOT NULL	-	Display title
5	content	TEXT	NULL	-	Transcript, note text or description
6	tags	TEXT[]	NULL	-	User and AI tags
7	embedding	VECTOR(1024)	NULL	-	BGE-M3 embedding (Qdrant mirror)
8	created_at	TIMESTAMP	NOT NULL	-	Capture time

Table: notification - broadcast messages from the admin console

#	Field Name	Data Type	Constraint	Key	Description
1	notification_id	SERIAL	NOT NULL	PK	Unique identifier
2	title	VARCHAR(200)	NOT NULL	-	Message title

#	Field Name	Data Type	Constraint	Key	Description
3	type	VARCHAR(30)	NOT NULL	-	System / feature / alert / policy
4	channel	VARCHAR(20)	NOT NULL	-	in-app / email / both
5	audience	VARCHAR(20)	NOT NULL	-	all / free / pro / team
6	status	VARCHAR(20)	NOT NULL	-	delivered / pending / failed
7	read_rate	DECIMAL(5,2)	NULL	-	Percentage read
8	sent_at	TIMESTAMP	NOT NULL	-	Send time

Further tables follow the same pattern: **task** (title, priority, category, due date, completed), **event** (calendar entries), **document** (file metadata), **person** (directory contacts with tags), **integration** (connected apps and sync state), **role** (role name and description), **permission** and **role_permission** (the roles matrix), **ai_usage_log** (model, tokens, latency per request) and **audit_log** (admin actions).

Schema as SQL (proposed)

```
CREATE TABLE role (
  role_id     SERIAL PRIMARY KEY,
  name        VARCHAR(60) NOT NULL,
  description VARCHAR(300) NULL
);

CREATE TABLE "user" (
  user_id    SERIAL PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) NOT NULL UNIQUE,
  plan       VARCHAR(20)  NOT NULL DEFAULT 'free',
  status     VARCHAR(20)  NOT NULL DEFAULT 'active',
  team       VARCHAR(60)  NULL,
  role_id    INT NOT NULL REFERENCES role(role_id),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE memory (
  memory_id SERIAL PRIMARY KEY,
  user_id   INT NOT NULL REFERENCES "user"(user_id),
  type      VARCHAR(20) NOT NULL,
  title     VARCHAR(200) NOT NULL,
  content   TEXT NULL,
  tags      TEXT[] NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE notification (
  notification_id SERIAL PRIMARY KEY,
  title    VARCHAR(200) NOT NULL,
  type     VARCHAR(30) NOT NULL,
  channel  VARCHAR(20) NOT NULL,
  audience VARCHAR(20) NOT NULL,
  status   VARCHAR(20) NOT NULL DEFAULT 'pending',
  read_rate DECIMAL(5,2) NULL,
  sent_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Entity Relationships

- Role -> User: one role is held by many users (user.role_id foreign key).
- User -> Memory: one user owns many memories (memory.user_id).

- User -> Task / Event / Document / Person: the same one-to-many ownership pattern.
- Role -> Role_Permission -> Permission: the roles matrix joins roles to granular permissions.
- Ai_Usage_Log and Audit_Log reference the acting user for accountability.

Mapping to the Current Implementation

In the delivered frontend, each table is represented by a typed array in the corresponding page module (for example users with plan, status and team fields in the admin users page), and every control on the page operates on that array exactly as it will operate on API data: filters select rows, stat cards aggregate columns, add and delete actions mutate the array, and the CSV export serialises it. Connecting the backend therefore means replacing each array's initialiser with a fetch call, with no changes to the interactive layer.

5.4 MODULE

Pathii is divided into twelve functional modules across the two panels. Each module is a self-contained page with its own data, state and handlers.

Module	Panel	Route	Purpose
Dashboard	User	/	Today's overview: stats, recent memories, tasks, events
Voice Capture	User	/voice-capture	Recording studio with waveform and recent captures
Memories	User	/memories, /memories-classic	Searchable, filterable memory library
Tasks & Schedule	User	/tasks	Priority tasks, checkboxes, timeline
Calendar	User	/calendar	Month grid with event management
Documents / People	User	/documents, /people	File directory and contact directory
Integrations / Insights / Settings	User	/integrations, /insights, /settings	App shortcuts, analytics charts, preferences
Admin Overview	Admin	/admin	Platform KPIs, recent users, activity
User Management	Admin	/admin/users	Accounts table: plans, statuses, teams, actions
Content Administration	Admin	/admin/memories, /admin/documents, /admin/tasks	Memory, document and task administration
AI & Analytics	Admin	/admin/ai-usage, /admin/analytics, /admin/reports	Usage analytics, platform charts, report exports
Ops & Governance	Admin	/admin/notifications, /admin/system-health, /admin/roles, /admin/settings	Broadcasting, monitoring, permissions, configuration

5.4.1 Dashboard Module

The landing page greets the user by name and summarises the day: four stat cards (total memories, active tasks, upcoming events, important people) with trend indicators, a hero capture card that links to the voice studio, a recent-memories list with per-type icons and actions, upcoming tasks with priority badges, and an events list. The header search filters the recent-memories list live. A "Switch to Admin Panel" shortcut cross-links the two interfaces.

5.4.2 Voice Capture Module

The voice studio provides a large record button with a recording state (timer, pulsing indicator), playback controls and a CSS waveform visualisation. Beneath it, quick-note and file-attach actions and a recent-captures list with tags. Starting a recording adds a new capture to the list when stopped; every capture card carries type, duration and timestamp fields.

5.4.3 Memories Module

The memories library is the platform's core retrieval surface. A search box filters by title; type pills (All, Voice, Notes, Documents, Images, and more) filter by record type; both conditions combine. Selecting a memory opens the detail panel with its transcript or description, tags and actions. The module exists in two approved designs (the primary at /memories and the classic at /memories-classic), both fully interactive. The add-memory action appends a new record and confirms with a toast.

5.4.4 Tasks & Schedule and Calendar Modules

The tasks page combines a filter tab bar (All, Events, Reminders), a task list with completion checkboxes, priority and category badges, and a side column with today's timeline. Checking a task strikes it through and updates the completed count. The calendar module renders a month grid with an events sidebar; the add-event action inserts a real event into the grid for the selected day.

5.4.5 Documents, People, Integrations, Insights and Settings Modules

Documents offers a grid of file cards with type filters and per-file download; People is a contact directory with tag pills and a detail inspector; Integrations lists connected applications with working toggle switches; Insights renders recharts visualisations (area, bar and donut) over the user's activity with stat cards; Settings manages profile, AI behaviour and privacy preferences with functional toggles and a data-export action.

5.4.6 Admin Overview and User Management Modules

The admin overview aggregates platform KPIs (users, memories, documents, tasks, uptime) with trend cards, a user-growth area chart, a memory-type donut, a recent-users table and an activity feed. User management is the console's workhorse: a five-user table with search, plan and status pills, cycling dropdowns, row checkboxes with select-all, per-row actions (suspend, activate, delete), an invite action that appends a new user, and a CSV export of the visible rows.

5.4.7 Content Administration Modules

Memory, document and task administration follow the shared table idiom with their own columns and filters - type and status pills for memories, category and type for documents, status tabs for tasks - plus stat rows derived from the data and export buttons. Add and cleanup actions mutate the tables with toast feedback.

5.4.8 AI Usage, Analytics and Reports Modules

AI usage charts model consumption across the planned model stack (requests, tokens, latency) with period dropdowns; platform analytics composes area, bar and donut charts with a seeded activity heatmap; reports lists generated reports with schedule entries and exports table data to CSV, toasting "No data to export" when a view is empty.

5.4.9 Operations and Governance Modules

The notification centre sends broadcasts (a send action appends a real row to the delivery table with status and read-rate fields); system health monitors services with status pills, a log feed and a heatmap; the roles module renders a permission matrix whose role columns derive from the roles table, with functional matrix toggles and role creation; settings exposes grouped configuration forms with save and reset actions and toast confirmation.

CHAPTER 6
SYSTEM DEVELOPMENT

6.1 INPUT AND OUTPUT DESIGN

Input design focuses on how users and administrators provide data to Pathii and how that data is validated before it is stored or displayed. Every input is a controlled React element: its value lives in page state, and every change re-derives the visible output deterministically.

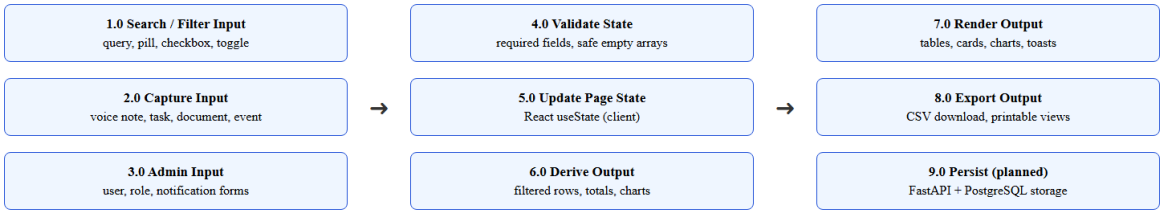


Fig. 6.1 - Input and output process flow

Process	Input	Output
1.0 Search / filter input	Query text, pill or tab selection	Controlled state update
2.0 Capture input	Voice recording, note, file, event form	New record appended to data
3.0 Admin input	User, role and notification forms	Validated record appended
4.0 Validate state	Current data array	Safe handling of empty arrays
5.0 Derive output	Data + search + filters	Visible-rows array, totals, charts
6.0 Render output	Visible rows	Tables, cards, charts, toasts
7.0 Export output	Visible rows	CSV download in the browser

Validation Rules

- Search and filters accept any text and never crash on empty results; an empty-state block renders in place of an empty list.
- Add actions create records with every field the row renderer reads, so new rows are immediately filterable, selectable and exportable.
- Table footers report the visible count when filters are active and the full count otherwise.
- Export actions guard against empty datasets with a toast instead of downloading an empty file.

Output Design

Output design presents results in a clear, consistent manner: filtered tables and lists re-render instantly; stat cards re-aggregate from the same derived array; toast confirmations appear bottom-right with a fade-in animation; CSV exports download through the browser's blob API; and

charts animate to their final state within two seconds. The design keeps a single source of truth per page - the derived visible-rows array - so no output can disagree with another.

6.2 SYSTEM STUDY

The system study examines the actors and their interactions with the platform. Two primary actors use the system: the User, who captures and manages personal knowledge, and the Administrator, who governs the platform.



Fig. 6.2 - UML Use Case Diagram: Pathii

Actor	Role	Use cases
User (primary)	Captures and manages personal knowledge	Capture voice / note / file; organise and tag memories; search and filter; manage tasks and calendar; view insights; manage settings
Administrator (primary)	Governs the platform	Manage users and roles; monitor system health; review analytics and reports; send notifications; configure integrations and settings
AI services (secondary, planned)	Processing actors	Transcribe audio (Whisper); embed memories (BGE-M3); answer queries (Qwen3 RAG); understand images (Qwen2.5-VL)

Component View

At the component level, every page module composes the same parts: a sidebar, a header with search, stat cards, content sections (lists, tables, charts), a toast layer and export helpers. The similarity is deliberate - a single mental model serves all twenty-four pages, which is what made the review-gated build process tractable and keeps new pages cheap to add.

6.3 SYSTEM TESTING

System testing evaluates the functionality, layout and reliability of the application. Because the system is a statically pre-rendered web application, testing combined build-time verification, automated DOM measurement, screenshot comparison and interactive browser testing.

Types of Testing Performed

Type of Testing	What was verified
1. Build / Type Testing	npm run build compiles every route with TypeScript strict mode; all 24 routes pre-render as static content; the build must exit 0 after every change.
2. Visual QA	Every route was screenshotted at 1920 x 1080 and compared against its approved design reference; per-page diffs confirmed transcription fidelity; all 24 routes passed.
3. Responsive Testing	A DOM sweep at 607 x 614 measured every element for horizontal clipping outside intentional scroll containers; zero offenders remained after fixes; desktop rendering was verified unchanged.
4. Hydration Testing	Grep audits guarantee no Math.random(), Date.now() or new Date() in any render path; seeded PRNGs produce identical server and client output; SSR HTML was byte-compared across builds.
5. Functional Testing	Search filters, filter pills, tabs, checkboxes, toggles, dropdowns, add and delete actions, CSV export and toasts were exercised in a real browser on every page.
6. Production Smoke Test	The production build served via npm start returned HTTP 200 on all 24 routes with correct titles, favicons and empty-state or seeded content.

Test Environment

Item	Details
Machine	Standard personal computer (Intel Core i5, 8 GB RAM)
Operating system	Windows 10 / 11
Software	Node.js 20+, Next.js 16.2.12, React 19.2.4, TypeScript 5, Tailwind CSS 4, recharts 3
Browsers	Chromium (automated) and everyday desktop browsers
Editor / tools	Visual Studio Code, Git, headless Chromium for screenshots

Test Scenarios

Scenario	Expected behaviour
Search with a query	List narrows to matching titles; empty result shows the empty state
Filter pill selection	Only rows of that type remain; stat cards and footer update
Add memory / user / notification	Row appended with all fields; toast confirms
Delete / suspend a row	Row removed or status flipped; toast names the record
Export with data	CSV downloads containing the visible rows
Export with no data	Toast says "No data to export"; no file downloads
Viewport 607 px	Page scrolls vertically; nothing clips; no visible scrollbars
Viewport 1920 px	Desktop dashboard layout renders exactly as designed
Unknown URL	Styled 404 page with links to both panels

Outcome of Testing

- All 24 routes pre-render statically and return HTTP 200 from the production server.
- Visual QA confirmed pixel-level fidelity with the approved design references.
- The responsive sweep reported zero clipped elements across all routes at 607 px width.
- Every interactive control produced its expected state change; toasts and exports verified.
- Hydration audits found no nondeterministic render paths; SSR output is reproducible.
- Seeded data is cross-consistent: identical names, counts and totals appear on every page that references them.

CHAPTER 7
SAMPLE OUTPUT

A) SCREENSHOT

Sample screens from the running production application - the user panel first, then the admin console. All screens were captured live from the production build at 1920 x 1080 with the seeded sample dataset.

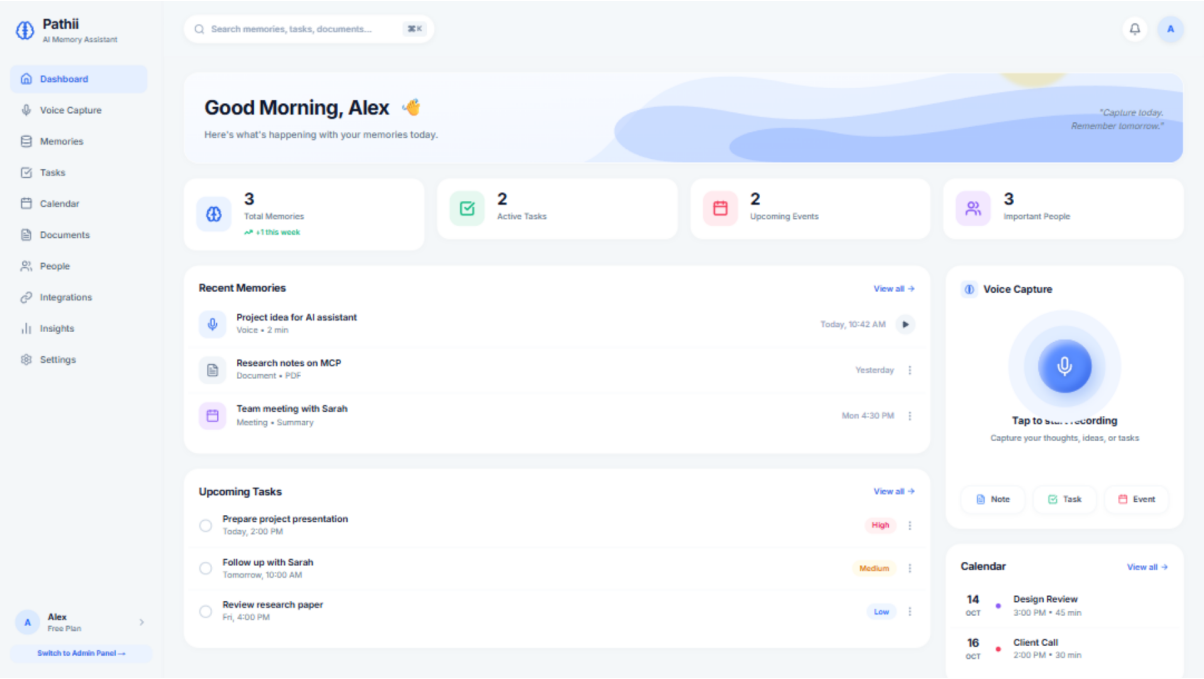


Fig. A.1 - User dashboard: stats, capture card, recent memories and tasks

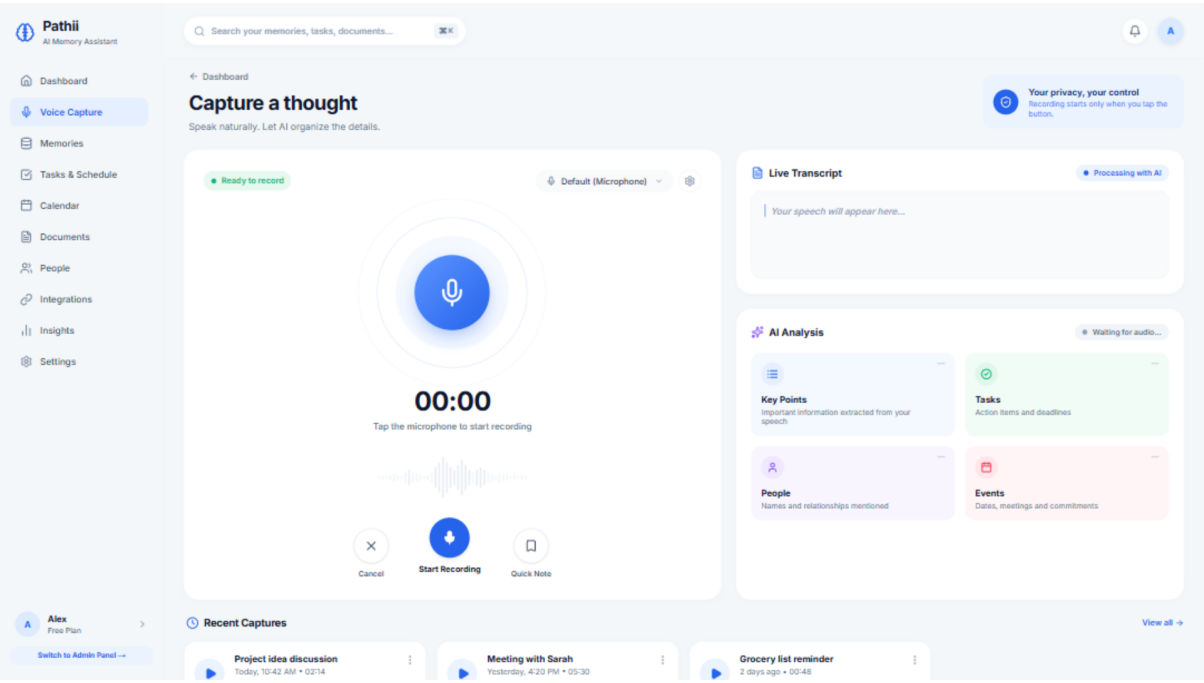


Fig. A.2 - Voice capture studio with waveform and recent captures

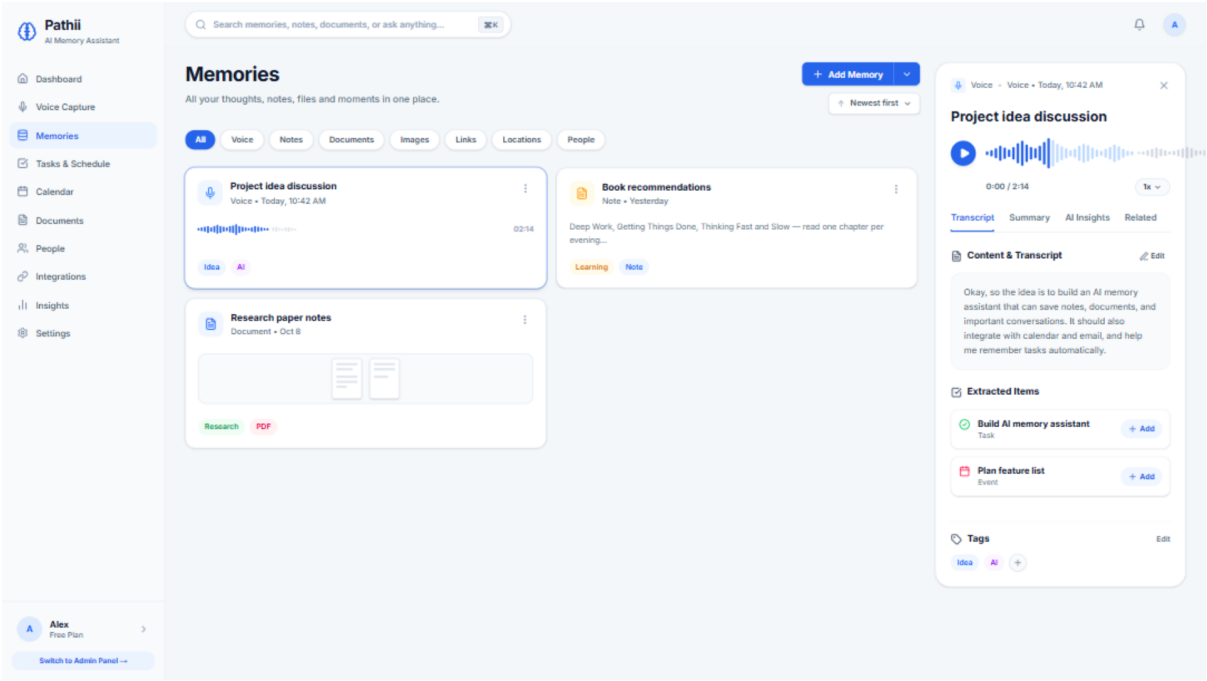


Fig. A.3 - Memories library with type filters and detail panel

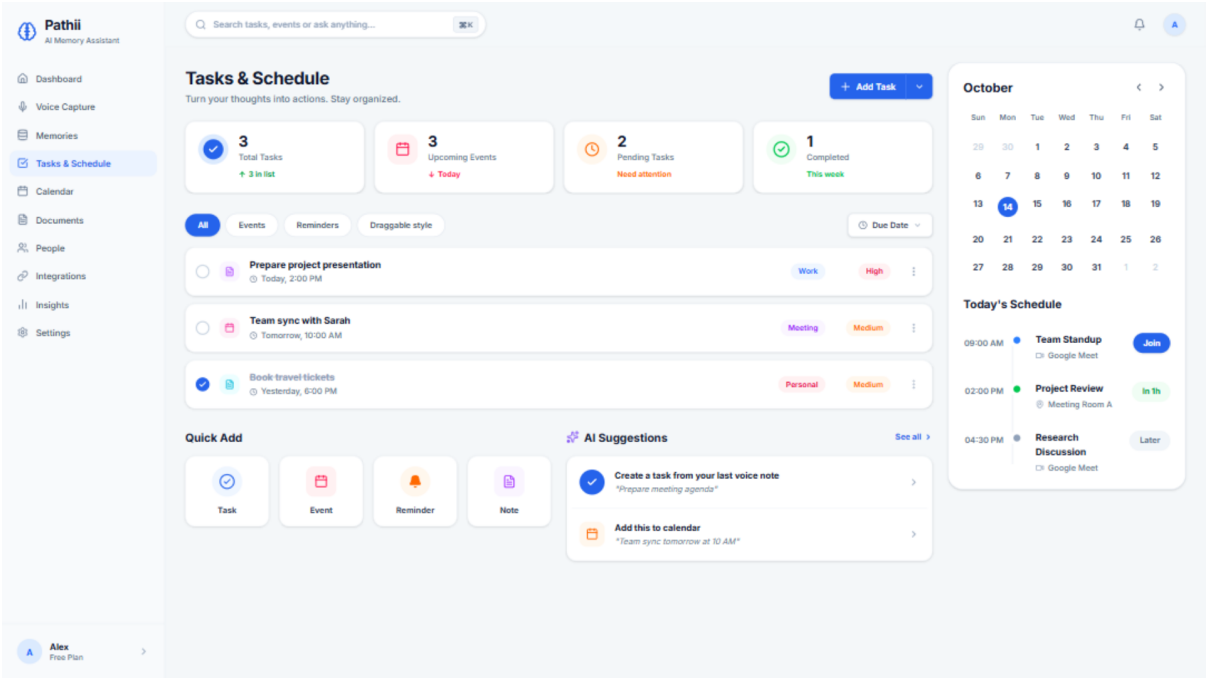


Fig. A.4 - Tasks and schedule with tabs and completion checkboxes

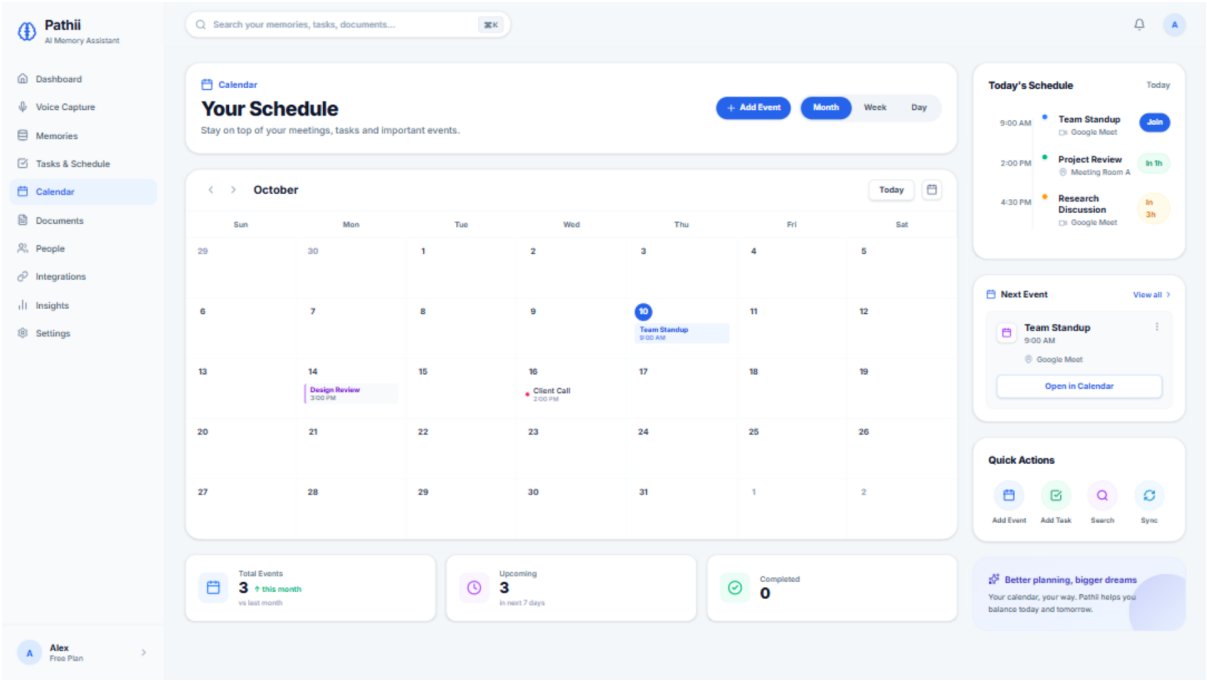


Fig. A.5 - Calendar month grid with events sidebar

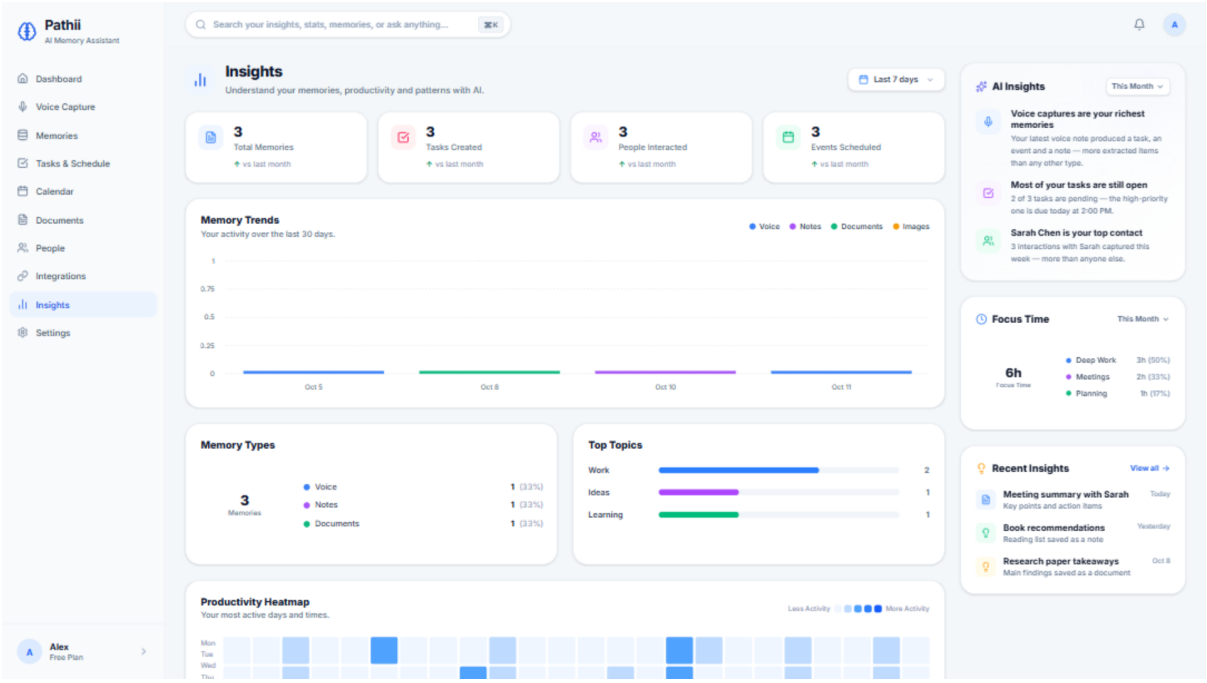


Fig. A.6 - Insights dashboard with charts

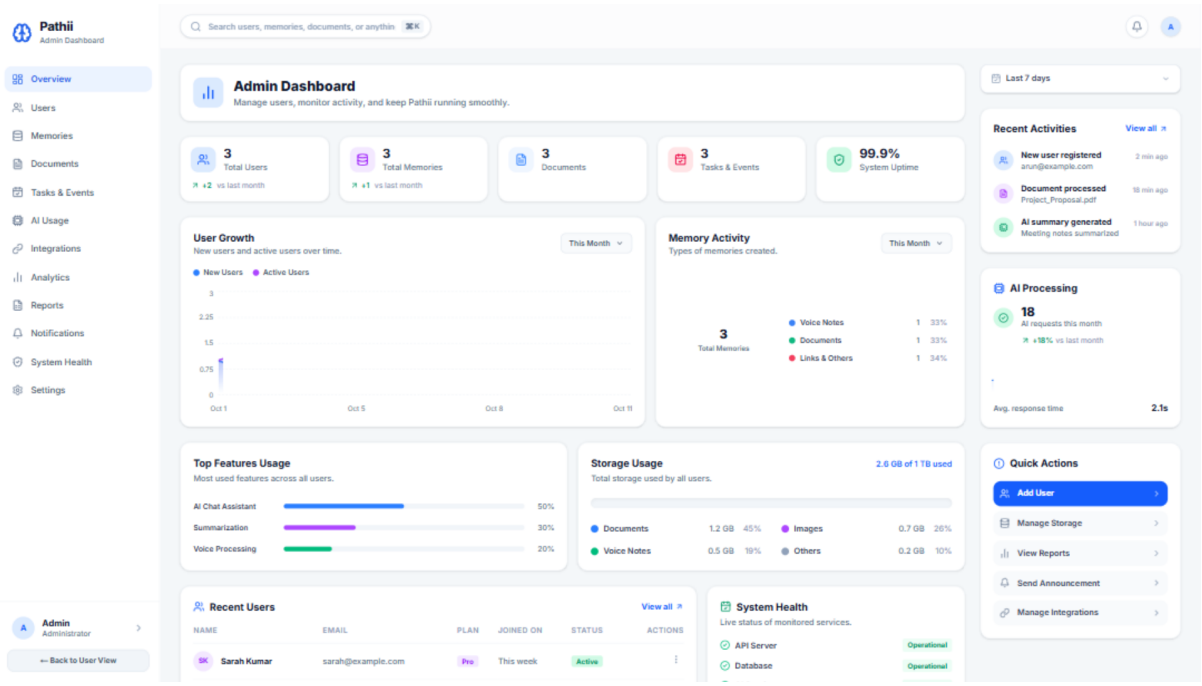


Fig. A.7 - Admin overview: KPIs, growth chart, recent users

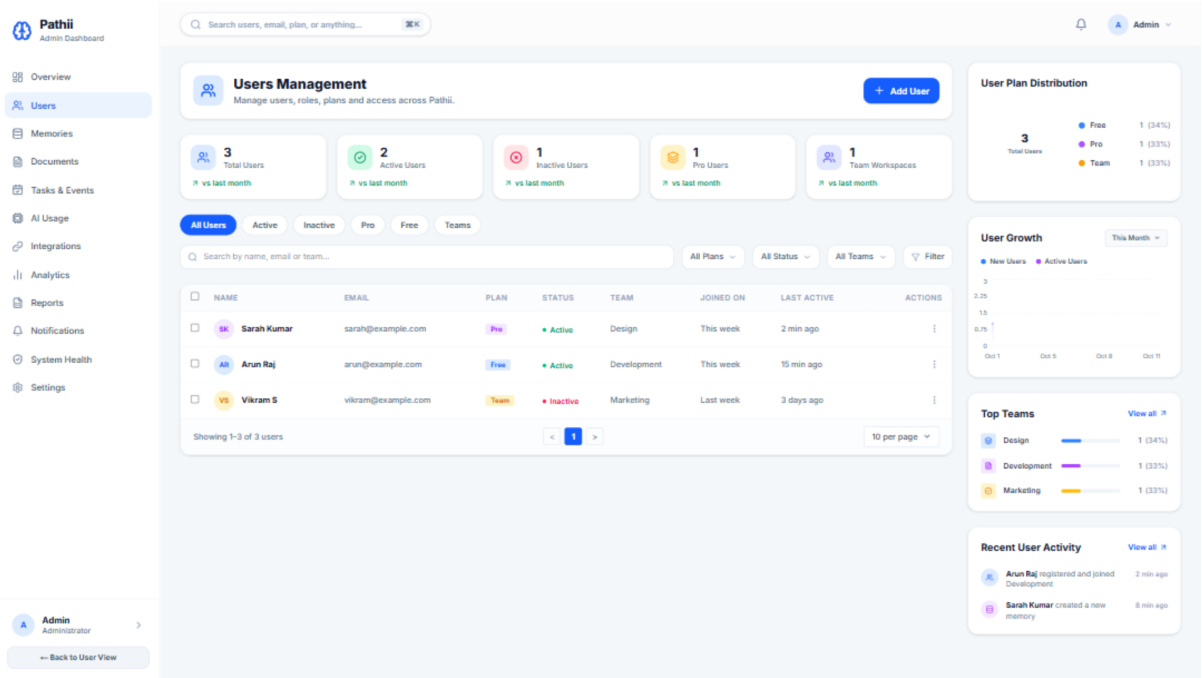


Fig. A.8 - User management table with filters and actions

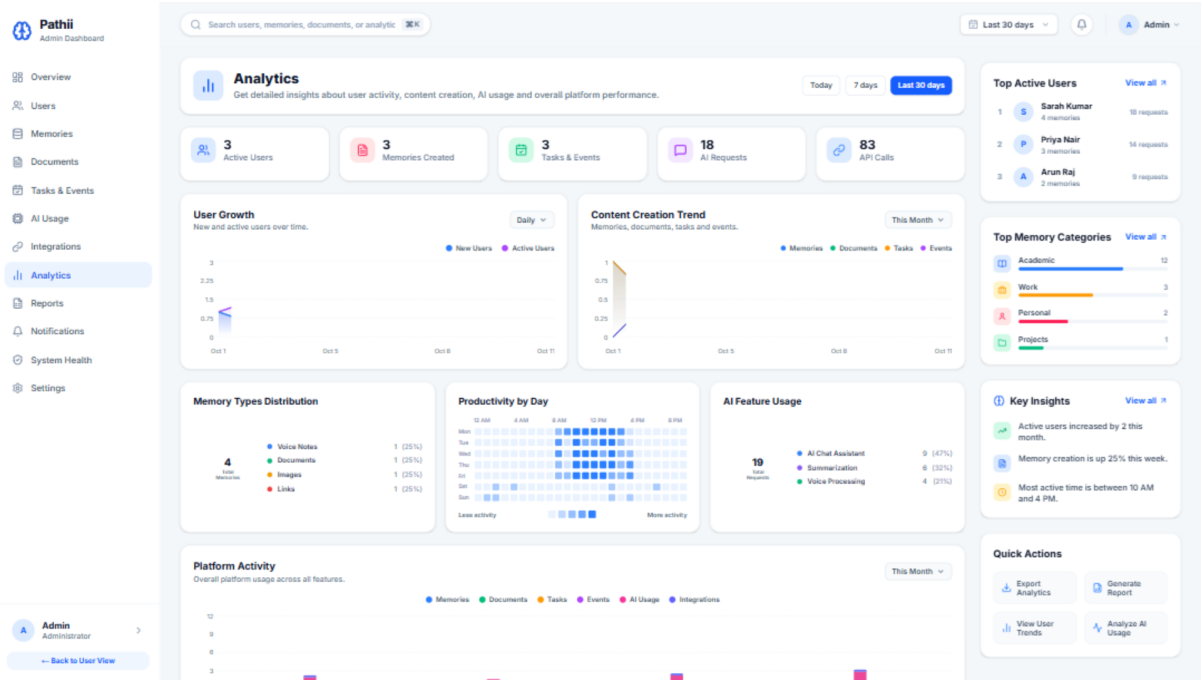


Fig. A.9 - Platform analytics with charts and heatmap

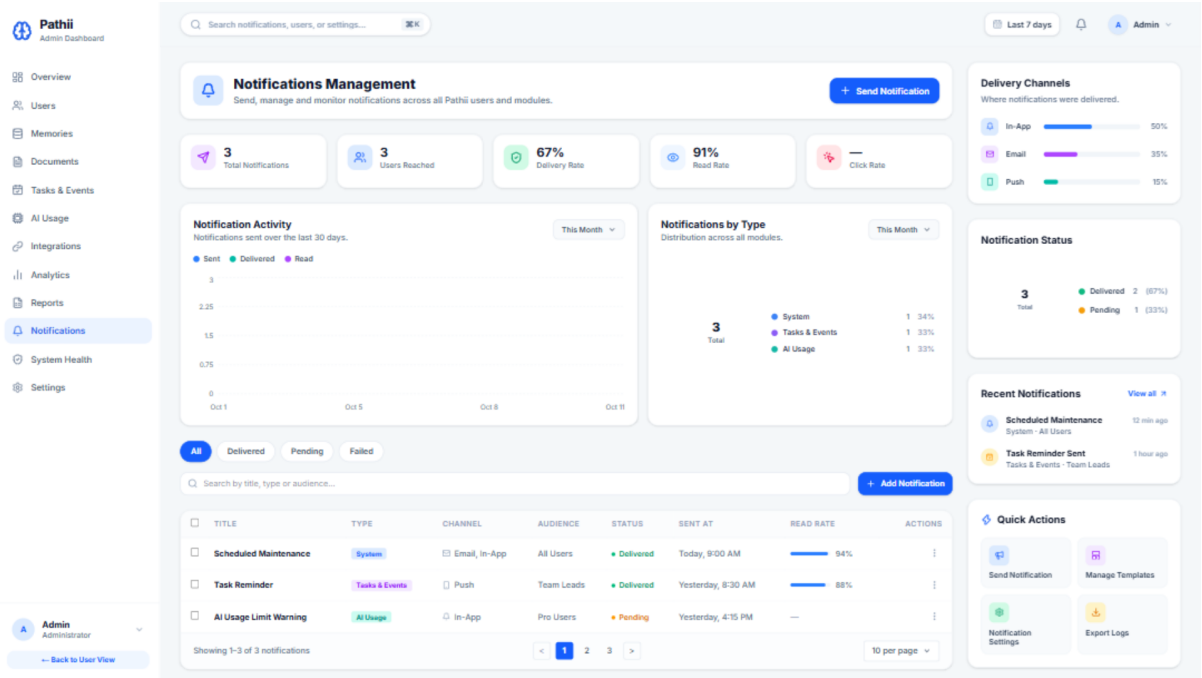


Fig. A.10 - Notification centre with delivery table

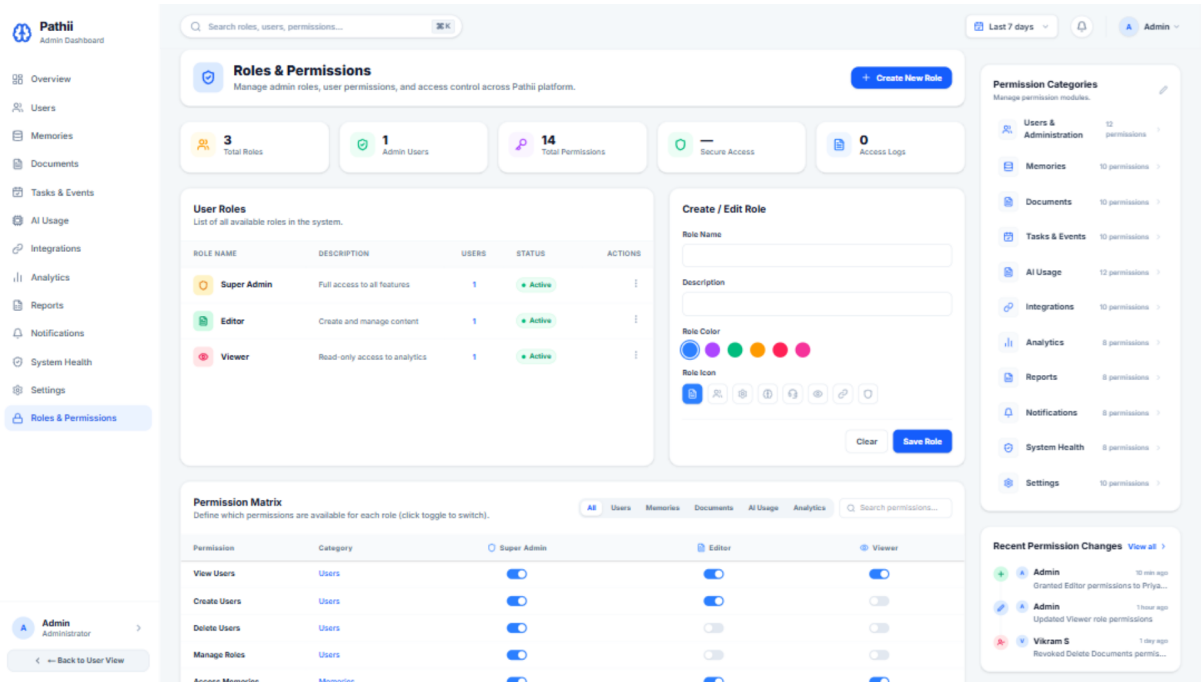


Fig. A.11 - Roles and permissions matrix

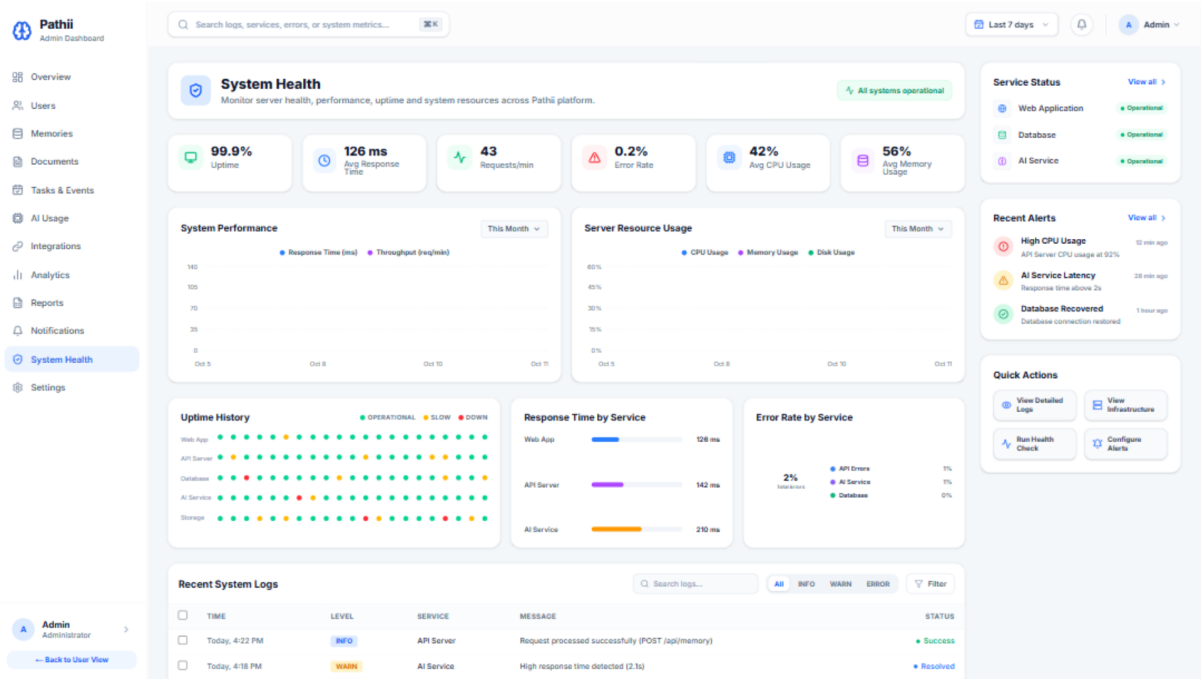


Fig. A.12 - System health monitor with service status and logs

B) SOURCE CODE

Key excerpts from the codebase, showing the navigation pattern, the deterministic data layer, the interactive filter layer and the export helper.

Sidebar navigation pattern (app/page.tsx, excerpt)

```
import NextLink from "next/link";

const SidebarItem = ({ icon: Icon, label, active, href }: any) => (
  <NextLink href={href}
    className={`flex items-center gap-3 px-4 py-3 rounded-xl cursor-pointer
```

```
      transition-colors mb-1 ${active
        ? "bg-[#E5EFFF] text-[#2563EB] font-semibold"
        : "text-slate-600 hover:bg-slate-50 font-medium"}`}>
    <Icon size={20} className={active ? "text-[#2563EB]" : "text-slate-500"} />
    <span className="text-[14.5px]">{label}</span>
  </NextLink>
);

// usage - one item per route, active only on the current page
<SidebarItem icon={Home}      label="Dashboard"     href="/"            active />
<SidebarItem icon={Mic}       label="Voice Capture" href="/voice-capture" />
<SidebarItem icon={Database}  label="Memories"      href="/memories" />
<SidebarItem icon={CheckSquare} label="Tasks"       href="/tasks" />
```

Seeded PRNG for hydration-safe charts (app/insights/page.tsx, excerpt)

```
// Replaces Math.random(): a seeded generator produces the same
// values on the server and the client, so pre-rendered HTML matches.
function seededRandom(seed: number) {
  return function () {
    seed |= 0; seed = (seed + 0x6D2B79F5) | 0;
    let t = Math.imul(seed ^ (seed >>> 15), 1 | seed);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

function generateHeatmap() {
  const rand = seededRandom(7);          // constant seed
  return services.map((service) =>       // 6 services x 35 cells
    Array.from({ length: 35 }, () => {
      const v = rand();                  // deterministic value
      if (v > 0.95) return "bg-rose-500/60";
      if (v > 0.85) return "bg-amber-400/50";
      return "bg-emerald-400/40";
    })
  );
}
```

Live search and type filter (app/memories/page.tsx, excerpt)

```
const [searchQuery, setSearchQuery] = useState("");
const [activeFilter, setActiveFilter] = useState("All");

const filterTypes: Record<string, string[]> = {
  All: ["voice", "note", "image", "document"],
  Voice: ["voice"],
  Notes: ["note"],
  Documents: ["document"],
  Images: ["image"],
};

const visibleMemories = memories.filter((m) => {
  const matchesSearch =
    m.title.toLowerCase().includes(searchQuery.trim().toLowerCase());
  const matchesFilter =
    activeFilter === "All" || filterTypes[activeFilter]?.includes(m.type);
  return matchesSearch && matchesFilter;
});
```

CSV export helper (app/admin/notifications/page.tsx, excerpt)

```
const exportRows = (rows: Notification[]) => {
  if (rows.length === 0) {
    showToast("No data to export");
    return;
  }
  const headers = ["Title", "Type", "Channel", "Audience", "Status", "Sent At"];
```

```
  const escape = (v: string) => `"${String(v).replace(/"/g, '""')}"`;
  const csv = [
    headers.map(escape).join(","),
    ...rows.map((r) =>
      [r.title, r.type, r.channel, r.audience, r.status, r.sentAt]
        .map(escape).join(","),
    ),
  ].join("\n");
  const blob = new Blob([csv], { type: "text/csv" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = "notifications.csv";
  a.click();
  URL.revokeObjectURL(url);
  showToast(`Exported ${rows.length} rows`);
};
```

Production configuration (next.config.ts)

```
import path from "path";
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  devIndicators: false,               // hide the dev overlay pill
  turbopack: { root: path.join(__dirname) },  // pin the workspace root
};

export default nextConfig;
```

CHAPTER 8
CONCLUSION

"Pathii: AI Memory Assistant" has been successfully designed and developed as a complete, production-ready web platform for multi-modal knowledge capture and administration. The system delivers twenty-four fully working pages across two panels - a personal workspace for capturing and recalling knowledge, and an administrative console for governing the platform - built with Next.js 16, React 19, TypeScript, Tailwind CSS and recharts.

Every screen went through a review-gated pipeline: pixel-exact transcription from an approved design, independent code review, visual comparison against the reference at 1920 x 1080, responsive verification down to a 607-pixel pane, hydration auditing, interactive wiring of every control, a production-readiness pass to an empty-by-default launch state, and finally the seeding of a small, cross-consistent sample dataset. The production build statically pre-renders all routes, serves with a single command, and contains no external asset dependencies.

Limitations

- Data lives in browser state; persistence requires the planned FastAPI backend.
- The AI pipeline (transcription, embeddings, semantic search, knowledge graph) is designed but not yet integrated.
- Authentication and multi-user sessions are not yet implemented; the two panels are open routes.
- Pagination on large tables is illustrative because the sample dataset holds one page of rows.

Future Scope

- FastAPI services with PostgreSQL, Qdrant, Neo4j and Redis behind the existing page contracts.
- Whisper transcription for voice memories and Qwen2.5-VL understanding for images.
- BGE-M3 embeddings with Qdrant for semantic search and Qwen3 RAG question answering.
- Authentication with roles enforced by the permissions matrix.
- Real-time synchronisation across devices and a mobile application.

In conclusion, the project demonstrates how a disciplined, design-driven frontend process can produce a complete production platform - and how every interactive behaviour, once built against in-page data contracts, transfers directly to a live backend. The result turns "capture everything, remember anything" from a tagline into a working system.

CHAPTER 9
BIBLIOGRAPHY

1. Lewis, P., Perez, E., Piktus, A., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems (NeurIPS), 2020.

2. Chen, J., Xiao, S., Zhang, P., Luo, Y., Lian, D., and Liu, Z. "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation." arXiv:2402.03216, 2024.

3. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. "Robust Speech Recognition via Large-Scale Weak Supervision." Proceedings of ICML, 2023.

4. Bai, S., et al. "Qwen2.5-VL Technical Report." arXiv preprint, 2025.

5. Vercel. "Next.js Documentation - App Router." Available at: https://nextjs.org/docs

6. Meta Platforms. "React Documentation." Available at: https://react.dev

7. Tailwind Labs. "Tailwind CSS Documentation." Available at: https://tailwindcss.com/docs

8. Recharts. "Recharts - A composable charting library built on React components." Available at: https://recharts.org

9. PostgreSQL Global Development Group. "PostgreSQL Documentation." Available at: https://www.postgresql.org/docs

10. Qdrant. "Qdrant Vector Database Documentation." Available at: https://qdrant.tech/documentation